Core Language Syntax

1 原子类别与记号

常量具有 `int`、`float`、`bool`、`event_id` 和 `pipeline_id` 五种类别，变量通过绑定指代相应的值。

元变量	所属集合	含义
x	Var	可数无限的局部变量名称集合
k	$\mathsf{Int} = \mathbb{Z}$	`int`：整数常量
r	Float	`float`：选定浮点格式的值集合
b	Bool	`bool`：$\{\mathsf{true}, \mathsf{false}\}$
ε	$\mathsf{EventId}$	`event_id`：未解释的事件标识集合
p, q	Pipe	`pipeline_id`：pipeline 实例标识集合

常量保留其类别标记，变量名称与常量相互区分。c 表示常量，v 表示操作数，C 表示命令，P 表示整个程序。下标用于区分同一类别的不同实例，例如 v_1, v_2 和 C_1, C_2。横线记号 $\bar{v}$ 是有限操作数序列 $v_1, \ldots, v_m$ 的缩写，其中 m 为非负整数；$m = 0$ 时，括号内为空，例如 $\mathsf{return}()$。

操作数在具体命令中可以指代数据、地址、事件标识、pipeline 标识或锁标识。地址和锁标识的编码由采用的抽象语义规定。

符号 $::=$ 表示语法定义，$|$ 分隔备选构造。语法产生式中以直立字体排印的命令名称为固定关键字。构造中的圆括号、方括号、花括号、逗号、分号、等号、$:=$ 和 $\|$ 均为语法符号；$\|$ 表示并发分支的分隔。

2 抽象语法

$$
\begin{aligned}
c &::= k \mid r \mid b \mid \varepsilon \mid p, \\
v &::= c \mid x, \\
C &::= \mathsf{return}(\bar{v}) \\
&\quad \mid \mathsf{let}\ x = v\ \mathsf{in}\ C \\
&\quad \mid \mathsf{let}\ x = \mathsf{compute}[p](\bar{v})\ \mathsf{in}\ C \\
&\quad \mid x := \mathsf{read}[p](v);\ C \\
&\quad \mid \mathsf{write}[p](v_1, v_2);\ C \\
&\quad \mid \mathsf{setflag}[p, q](v);\ C \\
&\quad \mid \mathsf{waitflag}[p, q](v);\ C \\
&\quad \mid \mathsf{lock}(v);\ C \\
&\quad \mid \mathsf{unlock}(v);\ C \\
&\quad \mid \mathsf{if}\ v\ \mathsf{then}\ \{C_1\}\ \mathsf{else}\ \{C_2\}, \\
P &::= \mathsf{run}\ \{C_1 \parallel \cdots \parallel C_n\}.
\end{aligned}
$$

n 为正整数；程序包含有限且非空的并发分支序列。省略号仅表示按所给下标展开的有限序列。每条命令中的后续程序 C 本身也是完整命令。

3 标注、绑定与良构条件

compute$[p]$、read$[p]$ 和 write$[p]$ 中，p 是 `pipeline_id` 常量，标注操作所属的 pipeline 实例。setflag$[p,q]$ 和 waitflag$[p,q]$ 中，p,q 均为 `pipeline_id` 常量：p 标注源 pipeline，q 标注目标 pipeline；两种构造的参数顺序一致。方括号中的 pipeline 标注在程序中静态确定。

静态配置声明可用的 pipeline 实例，给出执行单元标识集合 Unit 和映射 unit : Pipe $\to$ Unit。unit(p) 表示 pipeline p 对应的执行单元；不同 pipeline 可以对应同一个执行单元。在多核心配置中，pipeline 标识同时区分所属核心。

1. 程序中使用的 pipeline 实例均已在静态配置中声明，且具有确定的执行单元映射。
2. 在 let $x = v$ in C、let $x =$ compute$[p](\bar{v})$ in C 和 $x :=$ read$[p](v); C$ 中，x 绑定后续程序 C 中对应的变量使用。该绑定的作用域从 C 开始。
3. 每个并发分支内部的绑定位置使用互不相同的变量名，满足单次赋值（SSA）约定。所有变量使用均位于对应绑定的作用域内。不同并发分支分别具有自己的局部变量作用域。
4. if 的条件具有 `bool` 类别；setflag 和 waitflag 的操作数具有 `event_id` 类别。

绑定的作用域规定变量名称的可见性；异步结果是否已经可用由执行状态决定。良构条件允许表达缺失同步的程序，数据竞争、提前使用异步结果和死锁属于执行行为分析的对象。

4 命令含义

return$(\bar{v})$ 结束当前并发分支的命令执行，并给出输出值序列。return() 给出空序列。该命令不增加隐式等待。

let $x = v$ in C 将操作数绑定到局部变量 x，随后执行 C。

let $x =$ compute$[p](\bar{v})$ in C 在 pipeline p 对应的执行单元 unit(p) 上执行一次同步的抽象纯计算，将结果绑定到 x，随后执行 C。compute 是固定关键字，输入序列记录计算的数据依赖；计算关系由采用的抽象语义给出。

$x :=$ read$[p](v); C$ 在 pipeline p 上发起对 v 所指代地址的异步读取，并将 x 作为结果的绑定名称。发起读取后可以继续执行 C。使用 x 不触发自动等待。

write$[p](v_1, v_2); C$ 在 pipeline p 上发起向 v_1 所指代地址写入 v_2 所指代值的异步操作，随后继续执行 C。

setflag$[p,q](v); C$ 在源 pipeline p 上发起面向目标 pipeline q、以 v 所指代事件标识标记的非阻塞设置请求，随后继续执行 C。设置请求的完成条件由同步语义规定。

waitflag$[p,q](v); C$ 在目标 pipeline q 上等待来自源 pipeline p、以 v 所指代事件标识标记的同步条件满足，随后执行 C。

lock$(v); C$ 与 unlock$(v); C$ 分别获取和释放 v 所指代的互斥锁。获取条件尚未满足时等待；操作成功后执行 C。

if v then $\{C_1\}$ else $\{C_2\}$ 当 v 的值为 true 时执行 C_1，为 false 时执行 C_2。

run $\{C_1 \parallel \cdots \parallel C_n\}$ 各个并发分支访问共享内存，执行步骤可以交错。异步读写的发起与完成分别由执行语义描述；程序顺序本身不增加隐式等待。